I want you to design and build a production-quality Windows desktop application named “Lui-P File Manager.”

This is a simplified but powerful file-management application intended primarily for older users and users experiencing memory or cognitive difficulties. The interface must be extremely clear, forgiving, consistent, and easy to read. Safety, recoverability, accessibility, and simplicity are more important than squeezing many controls onto the screen.

Do not create a generic File Explorer clone. Build the specific four-column workflow described below.

Primary purpose

The program will help users locate files in several large source locations and copy those files into employee-specific destination folders located inside a locally synchronized Dropbox folder.

Some source folders are very large. For example, the Projects location can contain approximately 181 GB of data. Configured locations may include:

- Local folders and drives
- Mapped network drives
- UNC paths such as `\\SERVER\Projects`
- Locally synchronized Dropbox folders
- Other administrator- or user-configured locations

The application must index only configured source and destination locations. Search results must appear quickly without rescanning an entire 181 GB folder every time.

The normal workflow is:

1. Select a configured source location.
2. Find a folder or file using browsing, search, or date filters.
3. Review the selected source folder’s contents.
4. Select an employee’s destination location.
5. Review the destination’s existing contents.
6. Copy files or folders into that destination by dragging, using the context menu, or using Copy and Paste.
7. Clearly confirm success without requiring unnecessary confirmations.

Copy must be the default drag-and-drop operation. The original source file must remain in place.

Required technology

Use:

- C#
- Current stable .NET LTS release compatible with Windows 10 and Windows 11

- WPF for the desktop interface
- MVVM architecture
- SQLite for application configuration, search indexing, operation history, and recovery metadata
- Dependency injection
- Asynchronous file and indexing operations
- Windows-native file APIs where appropriate
- A standard Windows installer

Use a clean solution structure with separate projects or clearly separated layers for:

- User interface
- Application services
- File operations
- Search/indexing
- Logging and recovery
- Preview handling
- Windows shell integration
- Persistence/configuration
- Automated tests

Do not use Electron or create a web application disguised as a Windows program.

Main window layout

The main application must use four primary vertical columns.

Column A: Source locations

Column A displays configured source-location sections, such as:

- Projects (P:)
- Scans (S:)
- Data (Z:)

Each section must have:

- A clear name
- Its configured path
- A search box
- A button to clear the search
- Date-filter controls
- A scrollable list of matching folders and files
- A visible offline/unavailable indicator when a network location cannot be reached
- A retry option
- A friendly empty-state message

Both users and administrators can:

- Add source sections
- Remove source sections
- Rename source sections
- Reorder source sections
- Change the path assigned to a section

Removing a section from the interface must never delete its actual folder or files.

Each source section must remember its own:

- Search text
- Selected date type
- Date range
- Sort order
- Scroll position
- Selected item

Column B: Selected source contents

Column B displays the contents of the folder selected in Column A.

It must support:

- Files and subfolders
- Large, readable rows
- Familiar file-type icons
- Name
- Type
- Date modified
- Date created
- File size
- Sorting
- Multi-selection
- Keyboard selection
- Right-click context menu
- Drag-and-drop
- Copy and Paste
- Rename
- Create folder
- Move
- Delete
- Open
- Open with
- Properties
- Refresh
- Preview

The most commonly needed commands should be visible and plainly named. Do not rely exclusively on icons.

Column C: Destination locations

Column C displays configured destination locations, normally employee folders inside Dropbox. Example:

- Employee 1
- Employee 2
- Employee 3
- Employee 4

Each destination must have:

- A friendly display name
- The actual folder path
- A folder icon
- A clear selected state
- A visible unavailable or Dropbox-not-synchronized warning
- A right-click menu
- Drag-and-drop support

Double-clicking a destination must open that location in Windows File Explorer.

Users and administrators must be able to:

- Add destinations
- Remove destinations from the application
- Rename destination labels
- Change destination paths
- Reorder destinations

Removing a destination shortcut must not delete the destination folder.

Column D: Selected destination contents

Column D shows the current contents of the destination selected in Column C.

It must provide the same familiar file-list features as Column B, including:

- Files and folders
- Names, types, dates, and sizes
- Sorting
- Multi-selection
- Open
- Preview
- Rename
- Create folder

- Move
- Delete
- Copy and Paste
- Drag-and-drop
- Refresh
- Properties

This allows the user to see what is already in an employee's Dropbox folder before copying anything into it.

Resizable interface

The four columns must retain their assigned purposes, but users must be able to resize the columns.

Provide:

- Easy-to-grab column dividers
- Sensible minimum widths
- A Reset Layout command
- Automatic restoration of the last layout
- Good behavior on common screen resolutions
- Support for Windows display scaling

Do not allow a column to become so narrow that its basic controls are unusable.

Source and destination drag-and-drop

Dragging a file or folder from Column B to a destination in Column C or into Column D must copy it by default.

Dragging must provide obvious visual feedback:

- Highlight the intended destination
- Display a plain-language message such as "Copy 3 files to Sarah"
- Show the mouse operation as Copy
- Make invalid drop targets visually unavailable

Do not move or delete the source during the normal destination workflow.

Also support dragging files and folders:

- From Windows File Explorer into an application destination
- From the application into Windows File Explorer
- Between appropriate application locations
- Onto items in the bottom shortcut bar when applicable

For lengthy operations, display:

- Current file
- Overall progress
- Number completed
- Number remaining
- Transfer speed when useful
- Cancel button
- A completion summary

Canceling an operation must leave the filesystem in a known and safely recoverable state.

Copy conflict behavior

If a destination already contains an item with the same name, show a large, easy-to-understand dialog with:

- Replace
- Keep Both
- Cancel

The dialog must clearly display:

- Source filename
- Destination filename
- Source location
- Destination location
- Source modified date and size
- Destination modified date and size

“Keep Both” must generate a clear filename such as:

`Report (2).pdf`

For multiple conflicts, provide an optional “Do this for all remaining conflicts” checkbox.

Never replace an existing file silently.

If Replace is selected, preserve the previous destination file in the protected recovery area before replacing it.

File operations

Users should have the familiar file operations available in Windows File Explorer:

- Open
- Open with
- Copy

- Cut
- Paste
- Move
- Rename
- Delete
- Create folder
- Select all
- Properties
- Drag-and-drop
- Multi-select
- Keyboard shortcuts
- Send/copy to a configured destination

Use standard shortcuts such as:

- Ctrl+C
- Ctrl+X
- Ctrl+V
- Ctrl+A
- F2
- Delete
- Shift+Delete, with an especially strong warning
- Enter
- Alt+Enter
- Ctrl+Z, when an eligible application operation can be undone

Delete must always require an extra plain-language confirmation.

The delete dialog must display:

- What will be deleted
- Number of selected items
- Total size when available
- Current location
- Whether the action can be recovered
- Large Cancel and Delete buttons

Cancel should be the safe/default button.

Protected recovery and Undo

All eligible filesystem changes performed through Lui-P File Manager must be undoable from the Activity Log for 30 days.

Use a protected recovery store controlled by the application. Do not depend exclusively on the Windows Recycle Bin.

Before a destructive or overwriting operation, save the information and data required to reverse it.

Support safe undo for:

- Copy
- Move
- Rename
- Delete
- Replace/overwrite
- Create folder
- Batch operations

Examples:

- Undo Copy removes the destination copy only if it has not subsequently changed.
- Undo Move moves the item back to its original location.
- Undo Rename restores the original name.
- Undo Delete restores the deleted item to its original location.
- Undo Replace restores the prior destination version.
- Undo Create Folder removes the folder only if doing so will not destroy later user data.

Before undoing anything, validate that the current filesystem state still matches the logged operation.

Use file size, timestamps, identifiers, and hashes where appropriate. Do not blindly overwrite newer work during Undo.

If a safe automatic undo is no longer possible, explain why in plain language and offer safe alternatives. Never pretend an undo succeeded.

Recovery data must be retained for 30 days and then cleaned up automatically. Show recovery expiration dates in the Activity Log.

If an action involves a network location that is temporarily unavailable, retain the recovery record and allow the user to retry later.

Activity Log dashboard

Create a prominent Activity Log dashboard available from the main navigation.

Every user on a computer can view all actions performed through Lui-P File Manager on that computer and can initiate an eligible Undo.

Log at least:

- Date and time
- Windows username
- Computer name
- Operation type
- Source path

- Destination path
- File or folder name
- Number of items
- Total bytes
- Result
- Error details
- Whether Undo is available
- Recovery expiration date
- Unique operation ID
- Whether the operation was later undone
- Related original operation when an undo occurs

The dashboard must provide:

- Large Undo button for eligible entries
- Clear “Undo available until” text
- Search
- Date filtering
- User filtering
- Operation-type filtering
- Success/failure filtering
- Details panel
- Export to CSV
- Export to a readable support report
- Retry for failed or interrupted operations
- A visible explanation when Undo is unavailable

Group all files from one batch operation into one main log entry that can expand to show individual items.

The log database must be resilient to application crashes. Create the operation journal before beginning a mutating filesystem operation, update progress as it runs, and finalize the entry after completion.

Do not store only human-readable messages. Store structured operation data sufficient to verify and reverse each action safely.

Search and indexing

Index only configured Column A and Column C locations.

Supported locations include:

- Local drives
- Local folders
- Mapped drives
- UNC network shares
- Local Dropbox folders

Use SQLite with full-text search capabilities where appropriate.

Index at least:

- File or folder name
- Full path
- Parent path
- Extension
- Size
- Date created
- Date modified
- Whether the item is a file or folder
- Configured location ID
- Last indexed time
- Availability state

Search must:

- Return results as the user types
- Remain responsive with very large locations
- Search filenames and folder names
- Support partial matches
- Handle spaces and punctuation
- Clearly show the location of each result
- Never freeze the user interface

Provide date filtering by:

- Date modified
- Date created

Provide presets:

- Today
- Last 7 Days
- Last 30 Days
- This Year
- Custom Range
- All Dates

Remember the selected filter independently for each source section.

The indexing service must:

- Perform an initial background scan
- Show indexing progress without blocking normal use
- Incrementally update the index
- Use filesystem notifications when dependable
- Perform periodic reconciliation because network shares and Dropbox changes can be missed
- Detect renamed, moved, deleted, and newly created items

- Pause or back off when a network location is offline
- Resume automatically
- Provide a Rebuild Index command
- Provide per-location index status and last updated time
- Avoid excessive CPU, disk, and network use
- Exclude the application’s protected recovery storage from results

Search results must be verified against the filesystem before a destructive operation is performed, because an indexed item may have changed.

File preview window

Provide a separate resizable pop-out preview window.

Support previews for:

- PDF files
- Common images, including JPG, JPEG, PNG, GIF, BMP, TIFF, and WebP where practical
- Plain text and common text-based files
- Microsoft Word files
- Microsoft Excel files
- Common audio files
- Common video files

Preview behavior should be read-only.

Prefer safe Windows-native preview handlers or reliable supported components when practical. Do not require Microsoft Office merely to show a basic preview if a reasonable read-only alternative can be provided.

If a format cannot be previewed, show:

- File icon
- Filename
- Type
- Size
- Dates
- Open with Default Program button

Double-clicking a file anywhere in the application must open it using its associated Windows program.

The preview window should support:

- Previous and Next item
- Zoom for PDFs and images
- Page navigation for PDFs
- Basic media playback controls

- Open in Default Program
- Show in Folder
- File properties

Do not execute macros, scripts, or active content during preview.

Bottom shortcut bar

Create a large shortcut bar across the bottom of the main window.

The shortcut bar may contain:

- Files
- Folders
- Programs

Users must be able to:

- Add shortcuts through drag-and-drop
- Remove shortcuts without deleting the underlying item
- Rearrange shortcuts into any order using drag-and-drop
- Place shortcuts anywhere along the bar
- Rename the displayed shortcut label without renaming the real file
- Choose large or extra-large shortcut sizes
- Scroll when more shortcuts exist than can fit
- Right-click a shortcut for Open, Show in Folder, Rename Label, and Remove Shortcut

Double-clicking:

- A file opens it in its associated program.
- A folder opens it in Windows File Explorer.
- A program launches it.

Clearly distinguish removing a shortcut from deleting the real file.

Persist shortcut order and layout automatically.

If a shortcut target is missing or offline, do not remove it automatically. Display it as unavailable and provide Locate Target, Retry, or Remove Shortcut.

Windows File Explorer context-menu integration

Add a Windows File Explorer right-click command that lets users copy selected files or folders to one of the destinations configured in Lui-P File Manager.

The menu should appear as:

`Copy with Lui-P File Manager`

Its submenu should list configured destination names.

Requirements:

- Support one or multiple selected files and folders.
- Use the same copy engine, conflict handling, logging, recovery, and progress interface as the main application.
- Start the main application or a lightweight transfer window if it is not already running.
- Do not hard-code destination paths.
- Read destinations from that computer’s configuration.
- Handle long paths and UNC paths.
- Work on supported Windows 10 and Windows 11 versions.
- Use supported Windows shell-integration methods.
- Ensure uninstalling removes the context-menu integration cleanly.

Autosave and session restoration

The application must automatically save its state and reopen exactly where the user left off.

Persist:

- Window size and position
- Maximized state
- Column widths
- Source-section order
- Destination order
- Shortcut order
- Selected source
- Selected destination
- Selected folders
- Search text for each source
- Date filters
- Sort settings
- Scroll positions where practical
- Preview-window size and location
- Font and accessibility settings

Save state continuously with sensible debouncing, not only during normal application exit.

After a crash or forced shutdown, restore the last valid state without corrupting the configuration.

Do not restore a dangerous half-finished command automatically. Instead, detect interrupted file operations and show a recovery screen.

Accessibility and older-user design

The default design must use:

- Large readable text
- Minimum 16-point-equivalent main interface text where practical
- Large click targets
- Strong color contrast
- Clear selected states
- Familiar icons paired with text
- Plain-language labels
- Generous spacing
- Consistent control placement
- Visible hover and keyboard focus states
- Minimal animation
- No hidden gesture-only commands
- No tiny unlabeled toolbar buttons

Provide settings for:

- Font size
- Interface scaling
- Light theme
- Dark theme
- High-contrast theme
- Row height
- Icon size
- Confirmation-message duration
- Optional completion sound
- Reduced motion
- Reset layout
- Reset appearance settings

Support:

- Keyboard-only navigation
- Windows screen readers
- UI Automation accessibility names
- Logical tab order
- Tooltips
- Windows display scaling
- Touch-friendly targets where practical

Use short messages such as:

- “Copied 4 files to John.”
- “The Projects folder is not available. Check the network connection and try again.”
- “This file already exists in Sarah’s folder.”

- “Undo completed. The file was restored to its original location.”

Avoid technical error messages in the normal interface. Put technical details behind a Details button and in the diagnostic log.

Configuration

Each computer maintains its own configuration.

Store configuration and databases in appropriate Windows application-data locations. Do not store user-specific settings in the installation directory.

Provide a Settings panel for:

- Source locations
- Destination locations
- Shortcut appearance
- Fonts and scaling
- Themes
- Search/index status
- Recovery storage
- Recovery-space usage
- Logging
- Context-menu installation status
- Default copy behavior
- Notifications
- Diagnostics
- Application updates if implemented

Because both users and administrators may manage source and destination sections, do not require administrator privileges for ordinary configuration. Only request elevation for actions that actually require it, such as machine-wide shell integration or installation.

Validate configured paths and display friendly warnings without discarding the saved configuration when a drive is temporarily disconnected.

Reliability and file safety

File operations must be designed for real business data.

Requirements:

- Support long Windows paths.
- Support large files.
- Support large folder trees.
- Preserve timestamps where appropriate.
- Preserve file contents exactly.

- Handle read-only files.
- Handle permissions errors.
- Handle locked files.
- Handle insufficient disk space.
- Handle disconnected network shares.
- Handle Dropbox synchronization delays.
- Handle partial operations.
- Handle application crashes.
- Avoid loading entire large files into memory.
- Use buffered streaming.
- Calculate progress without freezing the interface.
- Never report success until the operation is actually complete.
- Verify copied data using size and optional hashing according to operation and performance needs.
- Never delete a source as part of the normal destination-copy workflow.
- Never silently overwrite an existing destination item.

For operations requested through the app, use the application's controlled file-operation service rather than bypassing logging.

Changes made by unrelated external programs cannot always be undone by Lui-P. State this limitation clearly in documentation and never imply otherwise.

Security

Implement reasonable local application security:

- Sanitize filenames and paths.
- Prevent path traversal.
- Validate every source and destination at operation time.
- Do not execute content during indexing or preview.
- Protect the recovery store from casual modification.
- Do not log file contents.
- Do not expose credentials.
- Never store Dropbox or network passwords.
- Use the current Windows user's permissions.
- Redact sensitive diagnostic information when exporting if appropriate.
- Protect the SQLite databases from concurrent-write corruption.

Diagnostics

Maintain a separate technical diagnostic log for troubleshooting.

Include:

- Application startup and shutdown
- Indexer activity
- Network-location availability

- Exceptions
- Failed previews
- Shell-integration events
- File-operation failures
- Recovery cleanup
- Database migrations

Use rolling log files with configurable retention. Avoid unlimited log growth.

The user-facing Activity Log and technical diagnostic log are separate concepts.

Installer and deployment

Create a professional installer supporting Windows 10 and Windows 11.

The installer must:

- Install the application
- Install required runtime components or publish self-contained
- Offer the Windows context-menu integration
- Create Start Menu shortcuts
- Register file/shell components correctly
- Support clean upgrades
- Preserve user settings during upgrades
- Support clean uninstall
- Remove shell-integration components during uninstall
- Never delete the user’s business files
- Clearly warn before removing unexpired recovery data during uninstall

Avoid requiring administrator privileges every time the application runs.

Application appearance

Create a polished, modern Windows interface—not a spreadsheet.

Use the supplied four-column concept as the workflow reference, but translate it into a proper Windows application.

Visual direction:

- Clean and professional
- Friendly rather than childish
- Large typography
- Light neutral surfaces
- Strong but restrained accent color
- Clear borders between columns
- Rounded corners used sparingly

- Obvious selected rows
- Familiar folder and file icons
- Plenty of breathing room
- Minimal clutter

The name “Lui-P File Manager” should appear clearly in the title bar and About screen.

Architecture expectations

Before writing all implementation code:

1. Restate the requirements in a structured specification.
2. Identify risky or technically difficult areas.
3. Propose the solution architecture.
4. Define the SQLite schema.
5. Define the operation-journal and Undo model.
6. Define the index-update strategy.
7. Define the Windows context-menu approach.
8. Produce a screen-by-screen UI plan.
9. Produce a phased implementation plan.
10. Identify any genuine blockers requiring my decision.

Do not ask questions already answered in this prompt.

If a small implementation detail is unspecified, choose the safest and simplest behavior, document the assumption, and continue.

Required screens

At minimum, implement:

1. Main four-column file-manager screen
2. Activity Log and Undo dashboard
3. File preview pop-out window
4. Settings screen
5. Source-location editor
6. Destination-location editor
7. Index-status screen
8. Copy/move progress window
9. Duplicate-file conflict dialog
10. Delete confirmation dialog
11. Undo confirmation and result dialog
12. Interrupted-operation recovery screen
13. First-run setup
14. About and diagnostics screen

Automated testing

Create meaningful tests for:

- Copy
- Move
- Rename
- Delete
- Replace
- Keep Both
- Create folder
- Undo Copy
- Undo Move
- Undo Rename
- Undo Delete
- Undo Replace
- Undo batch operation
- Modified-file safety checks before Undo
- Recovery expiration
- Network path unavailable
- Interrupted operation
- Duplicate conflicts
- Index additions, changes, renames, and deletions
- Date filtering
- Search
- Session restoration
- Database migration
- Long paths
- Large-file streaming
- Permission failures
- Insufficient disk space where practical

Use temporary test directories and never run tests against real user folders.

Acceptance criteria

The first production-ready version is acceptable only when:

- A user can configure source and employee destination locations.
- Large configured locations index in the background without freezing the interface.
- Search results appear quickly.
- Date-created and date-modified filters work.
- Selecting an item in Column A displays its contents in Column B.
- Selecting a destination in Column C displays its contents in Column D.
- Dragging from Column B to Column C or D copies by default.
- Original source files remain intact.
- Duplicate files produce Replace, Keep Both, or Cancel.

- All application file operations create structured log entries.
- Eligible operations can be safely undone from the Activity Log.
- Recovery copies are retained for 30 days.
- Delete always requires extra confirmation.
- PDFs, images, documents, text, audio, and video have useful preview behavior.
- Double-click opens files with their Windows-associated programs.
- The shortcut bar supports files, folders, programs, and free reordering.
- Windows File Explorer can copy selected items to configured destinations through its context menu.
- The application restores its previous layout and working state.
- The default interface is large, clear, and usable by older adults.
- The application works on Windows 10 and Windows 11.
- The installer, upgrade, and uninstall processes work cleanly.
- No common failure path silently loses or overwrites data.

Start by producing the structured specification, architecture, data model, Undo design, UI plan, project structure, and phased build plan. After that, begin implementing Phase 1 as a compiling, runnable solution rather than returning isolated code snippets.